Backpropagation and Optimizer Comparison

Deep Learning (2087101) — Short Report

1. Aim

To implement backpropagation from scratch for a one-hidden-layer MLP and compare SGD with momentum and Adam on a synthetic classification dataset.

2. Question 1: Backpropagation from scratch

A small regression dataset is used with two input features, four hidden neurons, and one output neuron. Sigmoid activation is used in the hidden and output layers. The loss is Mean Squared Error (MSE). Gradients are computed with the chain rule, and weights and biases are updated using standard gradient descent.

For output prediction ■ and target y, the loss is L = mean((■ − y)²). The output error is propagated backward through the output layer and hidden layer. Parameters are updated as: W = W − learning_rate × gradient.

3. Question 2: Optimizer comparison

A synthetic two-class dataset is generated using make_moons. Both experiments use the same training/test split and feedforward network. The loss is binary cross-entropy with logits. The optimizers are SGD with momentum (learning rate 0.03, momentum 0.9) and Adam (learning rate 0.001). Training loss is recorded for each epoch. Test loss, accuracy, elapsed time, and convergence epoch are printed.

4. Output logs and results

Run the notebook and fill this table using your actual console output. Do not use estimated values.

Experiment / optimizer	Final train loss	Test loss	Accuracy	Time (s)
SGD + momentum	0.065314	0.107641	96.00%	2.10s
Adam	0.069131	0.094656	95.00%	2.53s

5. Analysis

Backpropagation uses the chain rule to calculate gradients and update parameters. SGD with momentum uses information from previous gradients to smooth updates. Adam adapts updates using moving averages of gradients and squared gradients. Compare the actual loss curves and final values to decide which optimizer converged faster in this run; the outcome depends on hyperparameters and the dataset.

6. Conclusion

The first experiment demonstrates gradient calculation and parameter updates without a deep learning framework. The second experiment compares two common optimizers under the same network and data conditions. The generated output logs and convergence plot provide evidence for the comparison.

7. How to reproduce

Open the notebook in Google Colab or Jupyter, install numpy, matplotlib, scikit-learn, and torch if needed, then run all cells. Copy the actual output logs and plot into the final submission. Runtime depends on the hardware used.